

¿Qué es una condición de carrera?

Es una situación en la que dos o más hilos intentan acceder y modificar la misma variable global al mismo tiempo. El resultado final del programa depende de "**quién gane la carrera**" al ejecutar sus operaciones en la memoria RAM.

¿Por qué puede producir resultados diferentes al ejecutar varias veces el programa?

Porque en C, la línea **global_counter++** (o **global_counter--**) **no es una instrucción atómica** (no se ejecuta en un solo paso indivisible). Para el procesador, esa simple línea en C equivale a 3 pasos independientes:

1. **Leer:** Traer el valor de **global_counter** desde la memoria RAM.
2. **Modificar:** Sumarle o restarle 1 al valor que acaba de leer.
3. **Escribir:** Guardar el nuevo valor de vuelta en la memoria RAM.

El planificador de hilos del sistema operativo conmuta entre un hilo y otro en microsegundos impredecibles. Si el Hilo 1 lee **global_counter** (por ejemplo, con valor 20) y el sistema operativo lo pausa justo antes de guardar el resultado para darle turno al Hilo 2, ambos hilos habrán leído el valor 20. El Hilo 2 resta 1 (obtiene 19) y lo guarda; cuando el Hilo 1 se reanuda, suma 1 a su 20 original (obtiene 21) y lo escribe en la RAM. **El decremento del Hilo 2 se perdió por completo.**

Como el sistema operativo interrumpe los hilos en momentos distintos en cada ejecución, el número final de iteraciones que "chocan" cambia constantemente, produciendo resultados impredecibles.

¿Qué es una sección crítica y qué parte del código corresponde a ella?

Una **sección crítica** es el bloque de código donde se manipula un recurso compartido (como **global_counter**) y que debe ejecutarse con **exclusión mutua** (es decir, solo un hilo a la vez tiene permitido estar allí).

En tu código C, la **sección crítica** es exactamente la línea donde alteras la variable global, la cual está delimitada y protegida por las funciones del mutex:

En thread_routine:

```
C
// --- INICIO DE LA SECCIÓN CRÍTICA ---
pthread_mutex_lock(&mutex);

global_counter++; // <--- Sección crítica (Modificación del recurso compartido)

pthread_mutex_unlock(&mutex);
// --- FIN DE LA SECCIÓN CRÍTICA ---
```

En `thread_routine_two`:

C

```
// --- INICIO DE LA SECCIÓN CRÍTICA ---
```

```
pthread_mutex_lock(&mutex);
```

```
global_counter--; // <--- Sección crítica (Modificación del recurso compartido)
```

```
pthread_mutex_unlock(&mutex);
```

```
// --- FIN DE LA SECCIÓN CRÍTICA ---
```

El **`pthread_mutex_lock`** actúa como un cerrojo: el primer hilo que entra bloquea la puerta. Si el segundo hilo llega mientras el primero está ejecutando **`global_counter++`**, se queda esperando afuera hasta que el primer hilo ejecute **`pthread_mutex_unlock`**.